



國立臺灣科技大學 電子工程系

碩士學位論文

應用於 nFAPI 拆分架構在不可預測網路壅塞下之自
適應提前時槽機制

Adaptive Slots Ahead for nFAPI Split under
Unpredictable network congestion

研究生：徐銘鴻

學 號：M11302209

指導教授：鄭瑞光博士

中華民國一百一十五年六月一十五日

教授推薦書



碩士學位論文指導教授推薦書

Master's Thesis Recommendation Form



M11302209

系所： 電子工程系
Department/Graduate Institute Department of Electronic and Computer Engineering

姓名： 徐銘鴻
Name MING HONG HSU

論文題目： 應用於 nFAP 分離架構之吞吐量-延遲權衡的自適應時序控制
(Thesis Title) Adaptive Timing Control for Throughput-Latency Trade-off in nFAP Split

係由本人指導撰述，同意提付審查。

This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：

Advisor's Signature

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

日期：

Date(yyyy/mm/dd)

_____/_____/_____

論文口試委員審定書



碩士學位考試委員審定書

Qualification Form by Master's Degree Examination Committee



M11302209

系所： 電子工程系
Department/Graduate Institute Department of Electronic and Computer Engineering

姓名： 徐銘鴻
Name MING HONG HSU

論文題目： 應用於 nFAP1 分離架構之吞吐量 - 延遲權衡的自適應時序控制
(Thesis Title) Adaptive Timing Control for Throughput-Latency Trade-off in nFAP1 Split

經本委員會審定通過，特此證明。

This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會

Degree Examination Committee

委員簽章：

Member's Signatures

指導教授簽章：

Advisor's Signature

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

系所（學程）主任（所長）簽章：

Department/Study Program/Graduate Institute Chair's Signature

日期：

Date(yyyy/mm/dd)

_____/_____/_____

摘 要

在 O-RAN 架構中，多廠商 (multi-vendor) 互通性為核心目標之一。為驗證不同設備間的相容能力，需進行端對端 (end-to-end) 的整合測試。然而，端對端測試涵蓋的元件眾多，且各元件皆需個別配置，導致整體測試流程繁瑣，並耗費大量人力與資源。為解決此問題，本文運用 OAI 軟體聯盟 (OSA) 所開發的 OAI 中央單元/分散單元 (OAI CU/DU)，提出一套可自動化執行與商用 O-RAN 無線單元 (O-RU) 整合之端對端測試方法，以提升測試效率並降低操作負擔。



Abstract

In the O-RAN architecture, multivendor interoperability is one of the core objectives. To verify the compatibility between different devices, end-to-end integration testing is required. However, such testing involves numerous components, each requiring individual configuration, resulting in a complex and labor-intensive process. To address this issue, this study proposes an automated end-to-end testing approach by leveraging the OAI Central Unit/Distributed Unit (OAI CU/DU) developed by the OpenAir-Interface Software Alliance (OSA), integrated with a commercial O-RAN Radio Unit (O-RU), aiming to improve testing efficiency and reduce operational overhead.



誌

謝

I would like to express my deepest gratitude to my advisor, Professor Ray, for his dedicated guidance and patient instruction throughout the course of my research. His mentorship has been invaluable, allowing me to gain both academic insights and practical experience that greatly enriched my personal growth.

I am also sincerely thankful to my parents, whose unwavering support enabled me to pursue my master's degree without worry and focus wholeheartedly on my studies.

I would like to thank my friend—Tzu-Yu, Mei-Chun, Yu-Hsuan, Chih-Hsuan, and Ching-Han—as well as the group “Ba Gwai.” During times of stress, they were always there to listen, have fun, and help me unwind, serving as my warmest emotional outlet. My gratitude also goes to my badminton group and NTUT basketball friends, who consistently encouraged me to exercise whenever I felt lazy—otherwise, I surely would have gained much more weight.

In the laboratory, I was fortunate to join a team with a harmonious and collaborative atmosphere. The senior members generously shared their knowledge and guided us with care, while the junior members were always willing to help. I am especially grateful for the tremendous support during my oral exam preparation. Beyond academics, we also enjoyed many group activities together, such as Christmas gift exchanges and dinners in Gongguan, which not only strengthened our bonds but also filled my graduate life with joy.

I am also very thankful for the group of wonderful friends I made in the lab—Big Winnie, Joanna, JOJO, YH, ZX, Heidi, Kenny, Ming, Toby, Richard, Mira, Will, Little Winnie, and Nino. Thank you for always being there to chat, share daily life, and accompany me through most of my master ' s journey.

Lastly, I wish to thank my classmates Jerry, Joe, Wilfrid, Johnson, Sheryl, and Sylvia. From preparing midterm exams in the first year to oral exams in the second year, the mutual support and camaraderie we shared have become some of my most cherished memories.



目 錄

教授推薦書	ii
論文口試委員審定書	iii
中文摘要	iv
英文摘要	v
誌謝	vi
目錄	viii
圖目錄	xi
表目錄	xii
第一章 緒論	1
一、 研究背景與 O-RAN 架構演進	1
(一) Split Option 2	2
(二) Split Option 6	2
(三) Split Option 7.2	2
二、 基於 Split Option 6 之網路架構與挑戰	3
(一) FAPI 介面介紹	3
(二) nFAPI 介面介紹	4

(三)	切分技術綜合比較	4
三、	研究動機與問題定義	5
四、	相關研究	6
五、	論文貢獻	7
第二章	系統架構	8
一、	端到端架構	8
二、	O-DU High : MAC 與延遲管理器	9
三、	O-DU Low : 高層物理層與定時時窗	10
四、	關鍵驗證指標	11
第三章	所提方法	13
一、	延遲管理架構	13
二、	節點同步	13
第四章	實驗結果	18
一、	實驗情境	18
二、	情境 I : PNF 到達偏移量 (Δt_{arrive}) 之 EWMA 驗證 . . .	21
三、	情境 II : 性能基準測試	24
四、	情境 III : 網路擁塞下的穩定性	24
第五章	結論	27

參考文獻	28
----------------	----



圖 目 錄

2.1	系統架構與延遲管理機制。	9
4.1	rApp 自動化評估框架架構。	20
4.2	PNF 到達偏移量 (Δt_{arrive}) 之 EWMA 處理驗證與主動調 整。	22
4.2	PNF 到達偏移量 (Δt_{arrive}) 之 EWMA 處理驗證與主動調 整 (續)。	23
4.3	性能評估：MAC RTT 效率與系統吞吐量。	25
4.4	不同網路擁塞程度下的系統穩定性與吞吐量表現。	26

表 目 錄

1.1	5G RAN 底層切分技術綜合比較矩陣	5
4.1	測試平台節點配置	19
4.2	無線系統參數	19



第一章 緒論

一、研究背景與 O-RAN 架構演進

在第五代 (5G) 及即將到來的第六代 (6G) 行動通訊標準的架構演進中，無線接取網路 (Radio Access Network, RAN) 經歷了從傳統的封閉式單體基站 (Monolithic Base Station) 向高度虛擬化、雲端化及解耦化 (Disaggregated) 架構的根本性變革。傳統架構下，基頻處理與無線電射頻 (RF) 功能被緊密綁定於單一供應商的專有硬體設備中，這不僅限制了網路佈署的靈活性，亦大幅增加了營運商的資本支出與營運支出。

隨著雲端無線接取網路 (Cloud-RAN) 的興起，解耦化的部署方式逐漸普及，其將基頻處理單元 (BBU) 集中至遠端的運算池 (BBU Pool)，並將遠端無線電頭端 (RRH) 保留於邊緣站點，兩者透過通用公共無線電介面 (CPRI) 進行連接。然而，C-RAN 的架構仍受限於龐大的前傳網路 (Fronthaul) 頻寬需求與嚴苛的延遲限制。為解決上述瓶頸並徹底打破供應商鎖定 (Vendor Lock-in) 的困境，O-RAN 聯盟 (O-RAN Alliance) [1] 與第三代合作夥伴計畫 (3GPP) 共同推動了 RAN 的進一步解耦，將傳統的 BBU 與 RRH 在邏輯與物理上正式拆分為三個獨立的網路實體：中央單元 (Central Unit, CU)、分散單元 (Distributed Unit, DU) 以及無線電單元 (Radio Unit, RU) [2,3]。

在這種解耦模型中，「功能切分選項」(Functional Split Options) 成為了決定網路效能、硬體複雜度、頻寬消耗以及延遲容忍度的最關鍵設計核心 [4,5]。3GPP 標準化了多種功能切分選項，以適應多樣化的網路需求與佈署場景。

(一) Split Option 2

Split Option 2 是一種高層切分 (High Layer Split, HLS) 架構，其主要將 CU 與 DU 進行分離。在這種切分方式下，封包數據匯聚協定 (PDCP) 與無線電資源控制 (RRC) 等高層協定集中於 CU 運行，而較低層的無線電鏈路控制 (RLC)、媒介存取控制 (MAC) 以及實體層 (PHY) 則保留在 DU。這種架構對前傳網路的頻寬與延遲要求較為寬鬆，並且有利於集中化管理與資源池化，因此被廣泛認定為具備最佳成本效益比的標準配置，特別適用於非即時性的資料傳輸場景。

(二) Split Option 6



Split Option 6，又稱為 MAC-PHY 切分，將架構的切割點精準設立於 MAC 層與 PHY 層之間。在此選項中，DU 負責處理包含 MAC 層在內的上層協定，而 RU 則保留了完整的實體層 (PHY) 處理能力。由於前傳網路傳輸的是經過 MAC 層排程後的「MAC 協定資料單元 (MAC PDU)」，其頻寬需求會根據使用者的實際負載動態調整，而非受限於通道的最大頻寬。這種特性使得 Split Option 6 能在頻寬受限的傳輸網路中展現極大的彈性與優勢。

(三) Split Option 7.2

Split Option 7.2x (Low-PHY 切分) 則是將切分點設於實體層內部，目前為 O-RAN 聯盟所主推的低層切分 (Lower Layer Split, LLS) 標準 [6]。雖然將部分運算 (如 FFT/IFFT) 移至 RU 藉以減輕前傳壓力，但其前傳頻寬需求仍與系統頻寬及天線數量成正比。這種切分方式通常需要極高容

量的前傳網路支援，並對時序同步的要求極其嚴苛，常需仰賴硬體級的精確時間協定 (PTP) 以維持嚴格的相位同步。

二、 基於 Split Option 6 之網路架構與挑戰

本論文專注於 Split Option 6 的架構設計與應用。如前所述，採用底層拆分（如 Split Option 7.2）雖然可以將運算資源進一步集中，但需要更昂貴的 PTP 部署環境以及對低延遲要求極高的傳輸網路，這將導致高昂的資本與營運支出。相對地，選用將實體層全留（Monolithic PHY）於遠端節點的 Split Option 6 方案，能在資源需求與成本效益上取得最佳平衡。根據 Khan 等人 [7] 的研究，針對 5G NR 小型基地台的成本收益權衡分析指出，相較於嚴格的底層切分，將 MAC 層與 PHY 層分離的架構能在確保良好吞吐量 (Goodput) 的同時，顯著降低對前傳網路的頻寬需求 (Request Bandwidth)，從而達到降低整體部署成本的目標。因此，在非理想或資源受限的網路環境中，Split Option 6 成為了兼顧效能與成本的理想選擇。

(一) FAPI 介面介紹

在 5G 基地台的內部架構中，MAC 層與 PHY 層之間的通訊依賴於標準化的介面。小型基地台論壇 (Small Cell Forum, SCF) 推出的功能性應用平台介面 (Functional Application Platform Interface, FAPI) 便是為此目的而生 [8]。FAPI 提供了一套標準化的控制訊息與資料負載格式，使得來自不同供應商的 MAC 與 PHY 元件能夠互相協同運作。然而，傳統的 FAPI 設計假設 MAC 與 PHY 位於同一實體設備內，並且兩者透過記憶體共享 (Shared Memory) 的方式進行快速且低延遲的資料交換，這限制了

基站功能在地理位置上的解耦與分散部署。

(二) nFAPI 介面介紹

爲了克服 FAPI 在物理部署上的限制並實現跨網路的 Split Option 6 架構，SCF 進一步開發了網路功能應用平台介面 (network Functional Application Platform Interface, nFAPI) [9]。nFAPI 的核心創新在於將原本依賴記憶體共享的 FAPI 訊息，封裝成可以透過乙太網路 (Ethernet) 或 IP 網路傳輸的網路封包協定 [10]。這項技術突破賦予了虛擬化網路功能 (VNF) 極大的彈性，使得運行於通用伺服器 (COTS Server) 上的雲端化 MAC 功能，得以透過封包交換網路遠端控制實體節點 (PNF) 上的 PHY 設備。



(三) 切分技術綜合比較

爲更直觀呈現不同切分選項之間的差異，表 1.1 針對 Split 8、Split 7.2x 以及 Split Option 6 在前傳資料型態、頻寬需求及延遲限制等方面進行了綜合比較。

本論文著重在 nFAPI 的部署與最佳化，因爲它提供了極佳的網路靈活性，允許網路營運商將基站邏輯拆分並部署在兩臺獨立的機器上，甚至透過一般的封包交換網路進行連接。然而，這種物理上的拆分也帶來了極大的挑戰：在非確定性的網路環境下，如何維護兩端節點的「時間同步 (Time Synchronization)」以及進行精確的「延遲管理 (Delay Management)」成爲了確保系統穩定運作的首要難題。

表 1.1: 5G RAN 底層切分技術綜合比較矩陣

評估維度	Split 8	Split 7.2x	Split Option 6 (nFAPI)
協定切割邊界	RF / PHY	Low-PHY / High-PHY	MAC / PHY
前傳資料型態	時域 IQ 樣本	頻域 IQ 符號	MAC PDU (封包化)
標準化介面	CPRI / eCPRI	O-RAN WG4 eCPRI	SCF nFAPI
頻寬成長驅動	系統頻寬、天線數	頻寬、空間層數	實際使用者流量
延遲限制	$< 250\mu s$	$< 250\mu s$	1 – 3ms (HARQ 預算)
最佳佈署場景	C-RAN、專屬光纖	巨集基地台	小型基地台、室內專網

三、研究動機與問題定義

雖然 Split Option 6 具備靈活的定時調整能力，且對於非理想前傳網路具備較佳的容忍度，但在實務佈署中仍面臨嚴峻挑戰。隨著無線接取網路邁向虛擬化與雲端化，託管 MAC 層的功能通常以虛擬網路功能 (VNF) 的形式部署於通用處理器上。然而，在這種通用運算環境中，非確定性的作業系統排程抖動與處理延遲的不確定性，往往會對時序敏感的基頻處理造成衝擊 [11,12]。

特別是在現有的開源實作（如 OpenAirInterface）中，nFAPI 介面的定時控制多依賴於靜態配置的「提前時槽」(Slots Ahead)。這種硬編碼方式無法動態適應封包交換網路中瞬息萬變的傳輸延遲與網路抖動。當前傳網路發生突發性的擁塞或 VNF 運算環境出現延遲波峰時，排程指令與控制訊息極易錯過預定的時序期限，進而產生「Missing Slot Packet」錯

誤。這類錯誤不僅會導致通訊中斷，更限制了 VNF 在動態雲端資源環境中的彈性擴展能力。

因此，本論文的核心研究問題定義如下：在具備非確定性處理負載與網路抖動的雲端環境中，如何動態且精準地補償 nFAPI Split Option 6 架構中的端到端延遲，確保時序同步的穩定性？

四、 相關研究

近年來，開放式無線接取網路 (O-RAN) 架構的快速發展，促使許多研究致力於優化拆分架構 (Split Architecture) 的效能與網路延遲管理。例如，Villa 等人提出的 X5G 測試平台 [13] 展示了一種基於硬體加速的先進 5G O-RAN 部署方案。該研究透過將實體層 (PHY) 的龐大運算卸載至專用的 NVIDIA GPU 與 Mellanox 智慧網卡 (SmartNICs)，成功實現了商用級的峰值吞吐量。然而，這種追求極致效能的架構高度依賴昂貴的專屬硬體設備，且前提是必須運行於具備硬體級精確時間協定 (PTP) 同步的理想網路環境中。這些嚴苛的條件，大幅限制了其在充滿不可預測延遲與抖動的一般封包交換網路 (Packet-switched networks) 中的適用性與部署彈性。

爲了解決上述高硬體成本與嚴苛網路要求的限制，本論文針對非理想前傳網路 (Non-ideal Fronthaul) 環境，提出了一種具備高普適性的軟體解決方案。不同於 X5G 依賴硬體加速來克服延遲問題，也不同於依賴靜態配置來妥協延遲抖動的傳統作法，本論文針對 nFAPI 架構，設計了基於指數加權移動平均 (EWMA) 延遲管理的「適應性提前時槽控制機制」。該機制能夠在一般商用現成伺服器上運行，動態追蹤並過濾網路抖動，主動適應不可預測的網路壅塞。實驗結果表明，本論文的方法無需仰賴高階專

屬硬體或精準的時鐘同步設備，即可有效穩定 HARQ 的循環延遲 (RTT) 並維持系統吞吐量，為 O-RAN 的部署提供了更具成本效益且靈活的替代方案。

此外，目前學界對 O-RAN 延遲管理的研究也可分為規劃、編排與排程三個層次。現有文獻 [14–17] 雖然在各層次提出了優化方案，但大多聚焦於長時尺度的編排或特定硬體加速。本論文則補足了針對 Split Option 6 固有的微秒級相位同步與延遲挑戰的缺口，呈現出在軟體定義網路架構下實現動態時間同步的獨特學術貢獻。

五、 論文貢獻



針對上述問題，本論文的主要貢獻如下：

- **nFAPI Split Option 6** 的多維延遲量化分析：我們系統性地分析並量化了影響同步失效的非確定性因素，包括 Linux 核心排程抖動、VNF 處理負載以及網路佇列延遲。
- **自適應提前時槽 (Adaptive Slots Ahead)** 機制：我們提出了一種基於節點同步與延遲管理 (Delay Management) 的動態調整機制。透過 EWMA 模型平滑定時資訊以估計即時抖動趨勢，並動態調整提前時槽 (S_{ahead})，確保排程訊息的準時性。
- **端到端商用測試平台驗證**：我們在 OpenAirInterface 平台上實作了所提機制，並結合商用 UE 與 O-RU 進行驗證。實驗結果顯示，該機制能有效消除 Missing Slot Packet 錯誤，並在高度擁塞的環境下維持穩定的網路吞吐量與低時延特性。

第二章 系統架構

在深入探討系統細節之前，本研究先介紹所採用的系統框架。爲了將網路功能應用平台介面 (nFAPI) 與主流商用設備整合，本研究所提之系統採用了一種具韌性的兩階段分割架構。網路首先透過網路功能之間的 Split 6 nFAPI 介面進行連接，隨後轉換爲標準的 O-RAN Split 7.2x 介面以連接商用 Pegatron O-RU。圖 2.1 展示了此系統架構及其延遲管理機制。

一、 端到端架構



本系統建立了一個完整的 5G 端到端通訊鏈路，從核心網路延伸至使用者設備：

- **核心網路 (Core Network, CN)**：負責路由、對談管理以及使用者資料認證的核心元件。
- **O-RAN 中央單元 (O-CU)**：處理高層協定疊的邏輯節點，如無線資源控制 (RRC) 與封包資料匯聚協定 (PDCP)。它透過 F1 介面與分散單元連接。
- **O-RAN 分散單元 (O-DU)**：爲了優化計算性能與部署靈活性，系統將 O-DU 分爲 O-DU High 與 O-DU Low。O-DU High 以虛擬網路功能 (VNF) 的形式在虛擬機器上執行，而 O-DU Low 則以實體網路功能 (PNF) 的形式部署在專用硬體上。
- **交換器 (Switch)**：位於 VNF 與 PNF 之間的商用網路交換器。此中

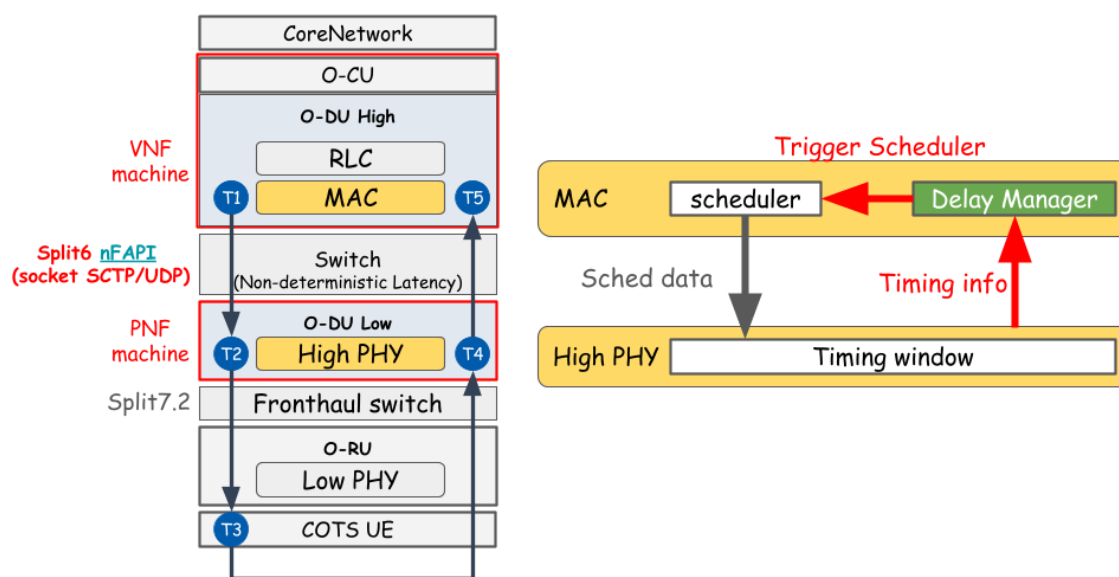


圖 2.1: 系統架構與延遲管理機制。

間層連接利用 nFAPI 協定。然而，穿越此封包交換網路不可避免地會產生非確定性的延遲。

- **O-RAN 無線單元 (O-RU)**：專用的硬體單元，負責低層物理層 (Low PHY) 處理、數位波束成型 (Digital Beamforming) 與射頻 (RF) 傳輸。
- **商用現成使用者設備 (COTS UE)**：作為網路終端接收器的標準商用智慧型手機或數據機。

二、 O-DU High : MAC 與延遲管理器

在 O-DU High 的媒體存取控制 (MAC) 層中，處理定時機制的核心組件包括排程器 (Scheduler) 與 nFAPI 延遲管理器 (nFAPI Delay Manager)：

- **排程器 (Scheduler)**：負責分配可用的無線資源 (如時頻資源塊)，

並決定下行/上行資料傳輸的確切優先順序與時序。

- **nFAPI 延遲管理器**：作為此延遲感知架構的關鍵控制單元。由於 VNF 與 PNF 之間的交換器存在不可預測且不穩定的延遲，延遲管理器持續接收來自低層物理層的定時回饋。它利用這些資訊動態計算網路抖動，並主動調整 提前時槽 (**slot-ahead, S_{ahead}**) 以觸發排程器。此計算確保排程指令 (Sched data) 能夠提前生成，以補償網路延遲。

三、 O-DU Low：高層物理層與定時時窗

O-DU Low 管理物理層的高層基頻處理 (High PHY)。在此層級中，最關鍵的同步機制是 nFAPI PNF 定時時窗 (Timing Window)：

- **定時時窗 (Timing Window)**：一個預定義且嚴格的時間區間。當 O-DU High 發送的排程訊息（特別是 DL_TTI.request）在網路中傳輸時，它必須準確地送達並恰好落在該時窗內。否則，高層物理層將拒絕該訊息，導致無法處理該時槽 (Slot) 的資料。
- **定時資訊 (Δt_{arrive})**：PNF 持續監控控制訊息相對於傳輸期限 (Deadline) 的確切到達偏移量。此偏移量表示為 Δt_{arrive} ，代表定時時窗內的剩餘時間裕度。PNF 定期將此資訊傳回 MAC 層的延遲管理器。這建立了一個閉迴路回饋控制系統，用以補償 VNF 與 PNF 機器之間不可預測的傳輸延遲變動。

四、 關鍵驗證指標

爲了評估所提演算法在非理想環境下的穩定性、同步精度與即時性能，本研究定義了以下關鍵驗證指標：

- **MAC 層 HARQ RTT ($T_5 - T_1$)**：專門測量 MAC 層的混合自動重傳請求 (HARQ) 迴路延遲。此指標精確反映了鏈路層的回饋效率與運作健康狀況。
- **VNF-PNF 下行延遲 ($T_2 - T_1$)**：測量從虛擬化機器 (VNF) 到實體機器 (PNF) 累積的單向下行延遲。此指標用於直接量化並分析由中間乙太網路交換器引入的延遲抖動影響。

註：RTT 代表往返時間 (Round-Trip Time)。爲了在非理想前傳條件下驗證分割架構的穩定性，本研究透過 Linux 流量控制 (TC) 在上行 ($T_5 - T_4$) 與下行 ($T_2 - T_1$) 路段進行網路延遲模擬。

爲了在現實網路環境中驗證所提出的性能優化，系統設計利用了商用級的部署框架：

- **O-RAN Split 7.2x 互操作性。**
儘管小型基地台論壇 (SCF) 已針對 MAC-PHY 分割 (Option 6) 標準化了網路功能應用平台介面 (nFAPI) [10]，商用級無線單元 (O-RU) 原生支援的是 O-RAN Split Option 7.2x。爲了與 OpenAir-Interface [18] 等開源協定疊達成無縫整合，本系統橋接了這些介面，以確保全規模的商用互操作性。
- 用於端到端驗證的混合系統架構。
爲了在完全商用的生態系統中驗證 nFAPI 的性能，本論文採用了

混合架構 (Split Option 6 + Split Option 7.2)。在此模型中，系統提供了一個轉換/互通層 (Translation/Interworking Layer)，使得由 nFAPI 驅動的 MAC/High-PHY 能夠成功地與商用 Pegatron O-RU 進行互動。此設置允許在真實世界的端到端環境中，使用商用現成 (COTS) 使用者設備來驗證 nFAPI 的性能。



第三章 所提方法

本章節詳細介紹所提出的動態定時調整演算法。為了解決 PNF 與 VNF 之間封包傳輸過程中的不穩定延遲，我們提出了一種「自適應提前時槽機制 (Adaptive Slots Ahead mechanism)」。

一、 延遲管理架構

本系統的定時控制聚焦於同步與延遲管理的邏輯層面，確保 VNF 端的提前時槽能夠靈活應對網路抖動，且不會引起預期外的震盪。此架構透過兩個主要的邏輯組件運作，兩者之間實現了無縫互動：

1. 節點同步 (Node Sync)：確保 VNF 與 PNF 之間的訊框與時槽邊界對齊。它監控定時偏移量以維持同步，從而吸收低層的傳輸抖動。
2. 延遲管理 (Delay Management)：聚焦於時槽 (Slot) 級別的延遲管理。它透過收斂優化演算法動態分析 PNF 回報的統計數據，以決定全域目標提前時槽 (S_{ahead})，確保排程指令始終符合 PNF 嚴格的定時時窗約束。

二、 節點同步

為了實現微秒級的相位對齊，系統採用了類似 IEEE 1588 (PTP) 的時間戳記交換機制。當 VNF 收到 UL_node_sync 訊息時，它根據四個時間戳記 (t_1 至 t_4) 計算瞬時定時偏移量 ($offset$)。為了避免瞬時突波，使用

指數加權移動平均 (EWMA) 模型對 $offset$ 及其變動量 ($error$) 進行平滑處理，產生平均偏移量 (μ_{offset}) 與平均絕對離差 (dev_{offset})，並利用平滑因子 α 與 β 進行調整。接著，直接從估計的抖動 (dev_{offset}) 導出動態同步閾值 ($Threshold$)。如果絕對偏移量超出此閾值，系統會根據時槽長度 ($slot_duration$) 將偏移量分解為時槽調整量 (S_{adj}) 與微秒級調整量 (μS_{adj})。這些參數隨後由 VNF 定時致動器 (VNF Timing Actuator) 使用，以精確對齊本地時鐘。此過程總結於演算法 1。



Algorithm 1 節點同步 (Node Sync)

Require: t_1, t_2, t_3 (PNF *UL_node_sync*), t_4 (VNF 本地接收時間戳記)

Ensure: S_{adj} (時槽調整量), $\mu_{S_{adj}}$ (微秒調整量)

符號說明：

offset (瞬時定時偏移量), μ_{offset} (EWMA 平均偏移量)

dev_offset (EWMA 平均絕對離差 / 抖動)

Threshold (動態同步閾值), α, β (平滑因子)

slot_duration (以 μs 為單位的時槽長度)

第一階段：時間戳記處理與迴繞保護

1: $d_1 \leftarrow t_2 - t_1$

2: $d_2 \leftarrow t_4 - t_3$

第二階段：偏移量與動態抖動估計

3: $offset \leftarrow (d_1 - d_2)/2$

4: $error \leftarrow offset - \mu_{offset}$

5: $\mu_{offset} \leftarrow \mu_{offset} + \alpha \cdot error$

6: $dev_{offset} \leftarrow dev_{offset} + \beta(|error| - dev_{offset})$

7: $Threshold \leftarrow dev_{offset}$

第三階段：相位對齊

8: **if** $|offset| > Threshold$ **then**

9: $S_{adj} \leftarrow \lfloor offset / slot_duration \rfloor$

10: $\mu_{S_{adj}} \leftarrow offset \pmod{slot_duration}$

11: 應用 S_{adj} 與 $\mu_{S_{adj}}$ 調整本地時鐘

12: **else**

13: 狀態 $\leftarrow$ 已同步 (**Synchronized**)

14: $S_{adj} \leftarrow 0, \mu_{S_{adj}} \leftarrow 0$

15: **end if**

16: **return** $S_{adj}, \mu_{S_{adj}}$

爲了對抗波動的網路延遲並維持排程的穩健性，本論文提出了一種動態的延遲管理策略。在收到定時統計數據（特別是 PNF 回報的最差情況延遲偏移量 Δt_{arrive} ）後，系統計算偏差 (*error*)，並結合受 Jacobson/

Karels TCP RTT 演算法啟發的 EWMA 模型，利用平滑因子 α 與 β 來估計平均延遲 (μ_{delay}) 與抖動偏差 (dev_{delay})。

在進行追蹤調整之前，演算法建立了多個恐慌準則 (**Panic Criteria**)。如果絕對偏差超過 $3 \cdot dev_{delay}$ ，則檢測到統計異常，表明出現了突然的延遲突波。如果 $\Delta t_{arrive} > slot_duration$ ，則發生嚴格違規，意味著封包延遲超過了時槽邊界。任一條件都會觸發快速攻擊恐慌模式 (**Fast Attack Panic Mode**)，將當前觸發定時 (S_{curr}) 激進地增加 1 個時槽來計算下一次觸發定時 (S_{next})，以確保連線連續性並減輕災難性的期限違規。

相反地，爲了在安全區域內最大化低延遲運行，採用了主動延遲縮減 (**Proactive Latency Reduction**) 機制。當延遲安全地低於絕對邊界 ($\Delta t_{arrive} \leq Safe_Margin$) 時，穩定計數器 ($count_{stable}$) 遞增。一旦此計數器超過隨 dev_{delay} 動態縮放的自適應穩定閾值 (N_{stable})，系統會謹慎地將 S_{curr} 減少 1 以確定 S_{next} 。這種非對稱響應（快速擴張、緩慢衰減）可防止乒乓震盪，同時最小化往返時間。最後， S_{next} 被限制在允許的 nFAPI 定時時窗內。調整策略濃縮於演算法 2。

Algorithm 2 延遲管理 (Delay Management)

Require: Δt_{arrive} (來自 PNF 的定時偏移資訊)

Ensure: S_{next} (下一次 MAC 排程的觸發定時)

符號說明：

S_{curr} (當前觸發定時), μ_{delay} (EWMA 平均延遲)

dev_{delay} (EWMA 平均偏差), $count_{stable}$ (穩定到達計數器)

N_{stable} (自適應閾值), α, β (平滑因子)

Safe_Margin (目標安全裕度), slot_duration (時槽長度)

第一階段：延遲統計更新

- 1: $error \leftarrow \Delta t_{arrive} - \mu_{delay}$
- 2: $\mu_{delay} \leftarrow \mu_{delay} + \alpha \cdot error$
- 3: $dev_{delay} \leftarrow dev_{delay} + \beta(|error| - dev_{delay})$

第二階段：異常檢測

- 4: $IsAnomaly \leftarrow (|error| > 3 \cdot dev_{delay}) \vee (\Delta t_{arrive} > slot_duration)$

第三階段：定時調整

- 5: **if** $IsAnomaly$ **then**
 - 6: $S_{next} \leftarrow S_{curr} + 1$
 - 7: $count_{stable} \leftarrow 0$
 - 8: **else if** $\Delta t_{arrive} \leq Safe_Margin$ **then**
 - 9: $count_{stable} \leftarrow count_{stable} + 1$
 - 10: **if** $count_{stable} \geq N_{stable}$ **then**
 - 11: $S_{next} \leftarrow S_{curr} - 1$
 - 12: $count_{stable} \leftarrow 0$
 - 13: **else**
 - 14: $S_{next} \leftarrow S_{curr}$
 - 15: **end if**
 - 16: **else**
 - 17: $S_{next} \leftarrow S_{curr}$
 - 18: $count_{stable} \leftarrow 0$
 - 19: **end if**
 - 20: 將 S_{next} 限制在允許的 nFAPI 定時時窗內
 - 21: **return** S_{next}
-



第四章 實驗結果

本研究在 OpenAirInterface (OAI) nFAPI 端到端環境中進行實驗，以驗證所提出的延遲管理方法。爲了確保評估的統計可靠性，報告中呈現的每個吞吐量 (Throughput) 值皆代表 180 個數據點的平均值 (3 次獨立試驗 $\times$ 每次試驗 60 個樣本)。

爲了驗證假設以及所提動態觸發排程演算法的有效性，實驗情境安排如下：第一、節概述了軟硬體環境與自動化評估框架。第二、節驗證了針對 PNF 到達偏移量 (Δt_{arrive}) 的指數加權移動平均 (EWMA) 機制。第三、節將高峰吞吐量與 MAC 層往返時間 (RTT) 的性能與靜態基準進行基準測試。第四、節評估了所提方法在不同網路擁塞程度下的穩定性。



一、實驗情境

測試平台環境連接了一系列高效能伺服器、無線單元與商用智慧型手機。端到端網路部署的詳細軟硬體配置如下表所示。

爲了確保自動化測量與分析具備一致的數據品質，本研究開發了一個自動評估 rApp，運行於 O-RAN 服務管理與編排 (SMO) 框架下的非即時無線存取網路智慧控制器 (Non-RT RIC) 上 [27,28]，如圖 4.1 所示。此 rApp 可自動啓動 VNF 與 PNF。一旦 gNB 成功建立，rApp 會自動透過 SSH 連接至控制電腦，利用 Android 偵錯橋接器 (ADB) [29] 切換 UE 的飛航模式，隨後透過 SSH 在核心網路伺服器上啓動 iPerf [30] 伺服器。接著，它會自動執行一系列預定的 iPerf 測試案例。最後，rApp 透過 O1 SFTP 收集所有日誌檔案，並利用 Python 腳本自動繪製後續的數據分析

表 4.1: 測試平台節點配置

單元	規格
核心網路 (Core Network, CN)	Open5GS [19]
軟體協定疊 / 分支	OpenAirInterface [20] (2026.w13)
VNF 伺服器	Intel® Xeon® Gold 6226R [21] 搭配 NetXtreme BCM5719 網卡 [22,23]
PNF 伺服器	Intel® Xeon® Gold 6433N [24] 搭配 Intel I210 網卡 [23,25]
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
商用現成設備 (COTS UE)	Samsung Galaxy S20 [26]

表 4.2: 無線系統參數

參數	設定
子載波間隔 (Subcarrier Spacing)	30 kHz
TDD 模式 (TDD Pattern)	3D1S1U
MIMO / 埠數 (MIMO / Ports)	4-layer / 4T4R
前傳介面 (Fronthaul)	O-RAN F release

圖表。在實驗過程中，我們也使用 tcpdump [31] 進行封包側錄，並利用 Linux PTP [32] 及 TSN 時間同步機制 [33] 確保網路節點間的基準時序，底層資料傳輸則依賴 DPDK [34] 進行封包加速。此框架的一個關鍵能力是能透過 Linux 流量控制 (TC)，在 MAC 回饋鏈路 ($T_5 - T_4$) 與前傳路段 ($T_2 - T_1$) 自動配置網路延遲模擬，這對於驗證解耦式架構在非確定性延遲下的穩定性至關重要。

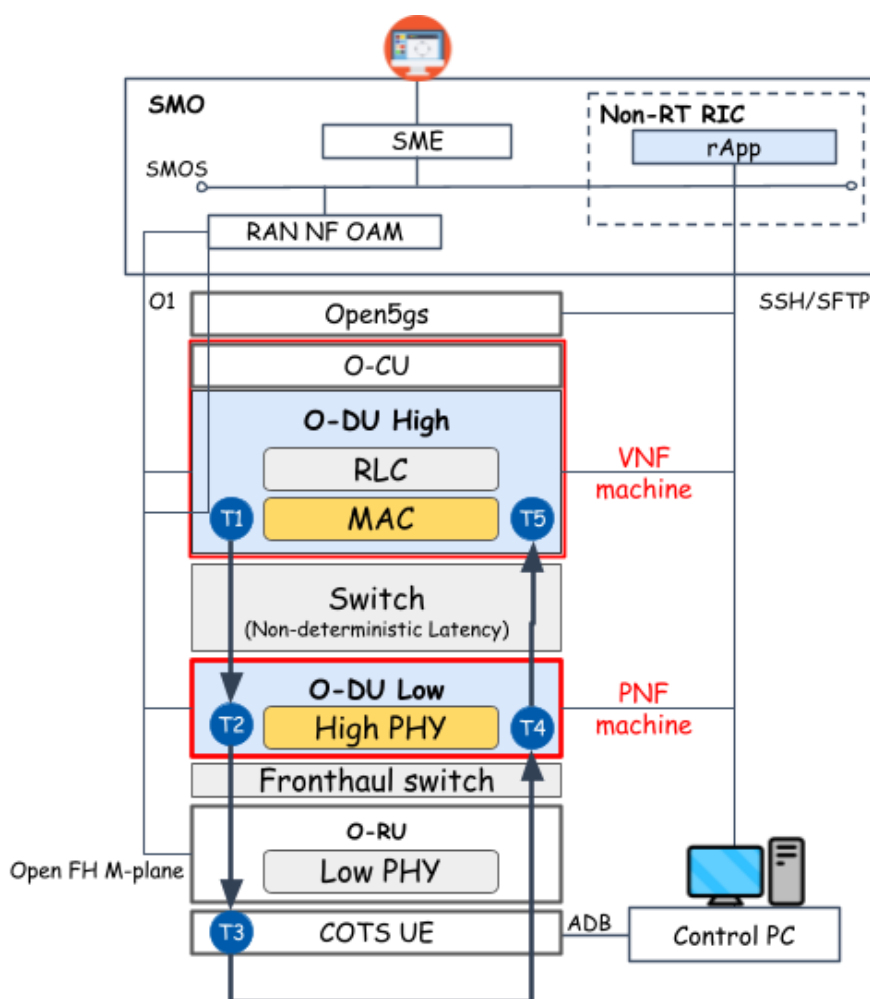


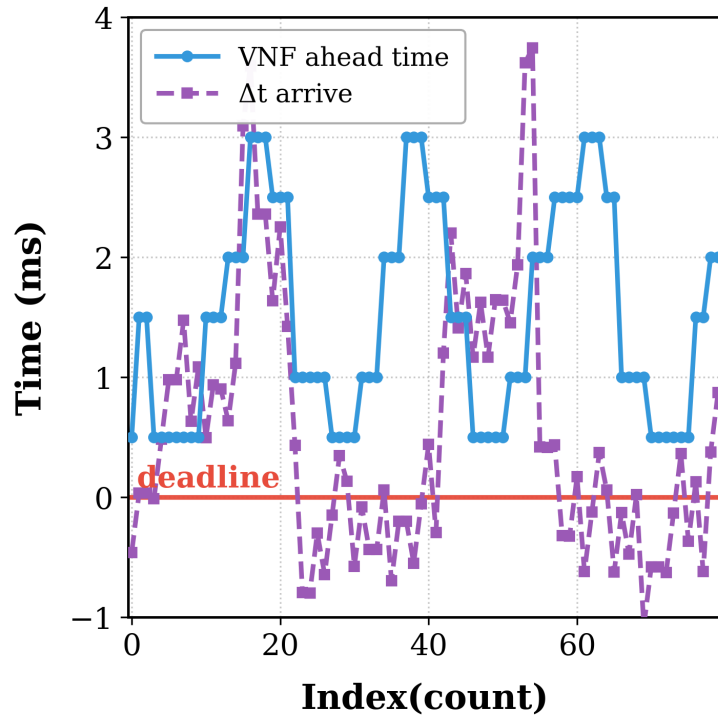
圖 4.1: rApp 自動化評估框架架構。

二、 情境 I : PNF 到達偏移量 (Δt_{arrive}) 之

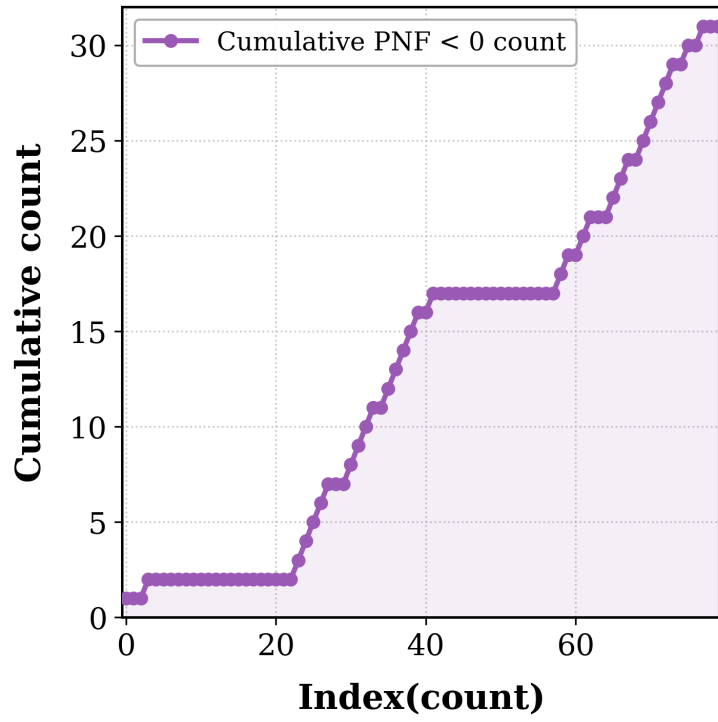
EWMA 驗證

此情境驗證了應用指數加權移動平均 (EWMA) 來平滑 PNF 到達偏移量 (Δt_{arrive}) 的有效性。由於網路抖動與處理延遲，從 PNF 接收到的原始定時偏移量可能會展現出高變異性。圖 4.2 展示了 EWMA 處理的影響。在不使用 EWMA 的情況下 (圖 4.2a)，到達偏移量波動劇烈，這可能導致不穩定的排程決策。透過應用 EWMA (圖 4.2b)，高頻抖動被有效過濾，為定時調整演算法提供了一個穩定的基準。此外，如圖 4.2c 與圖 4.2d 所示，平滑後的數據使系統能夠相對穩定地進行判斷。當 Δt_{arrive} 開始呈現下降趨勢時，演算法能夠穩定地識別此趨勢，並主動提前更多的時間 (S_{ahead})，以維持同步的穩定性。



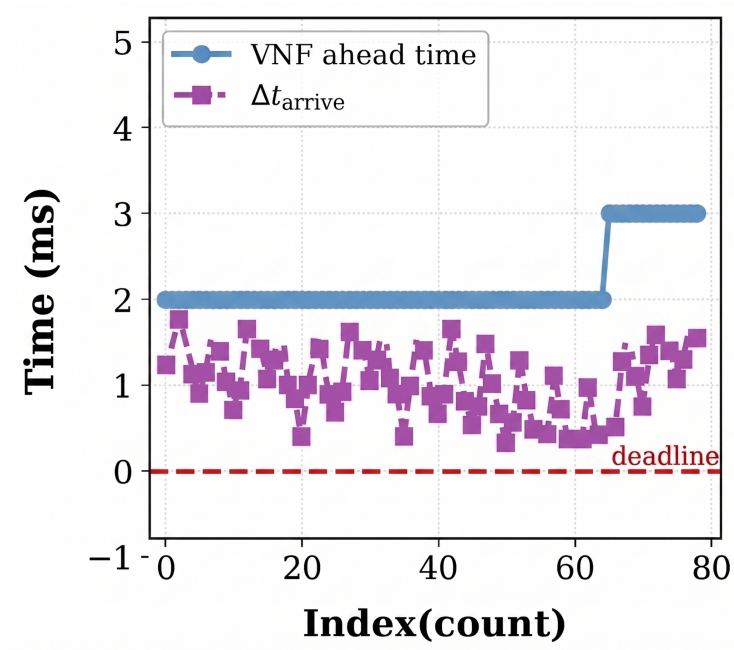


(a) 未應用 EWMA

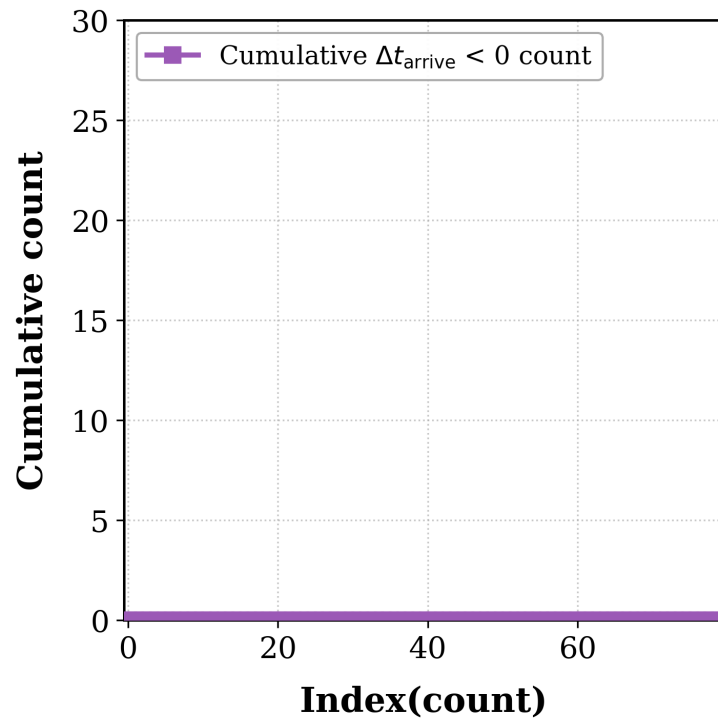


(b) 已應用 EWMA

圖 4.2: PNF 到達偏移量 (Δt_{arrive}) 之 EWMA 處理驗證與主動調整。



(c) 趨勢觀察



(d) 主動提前調整

圖 4.2: PNF 到達偏移量 (Δt_{arrive}) 之 EWMA 處理驗證與主動調整 (續)。

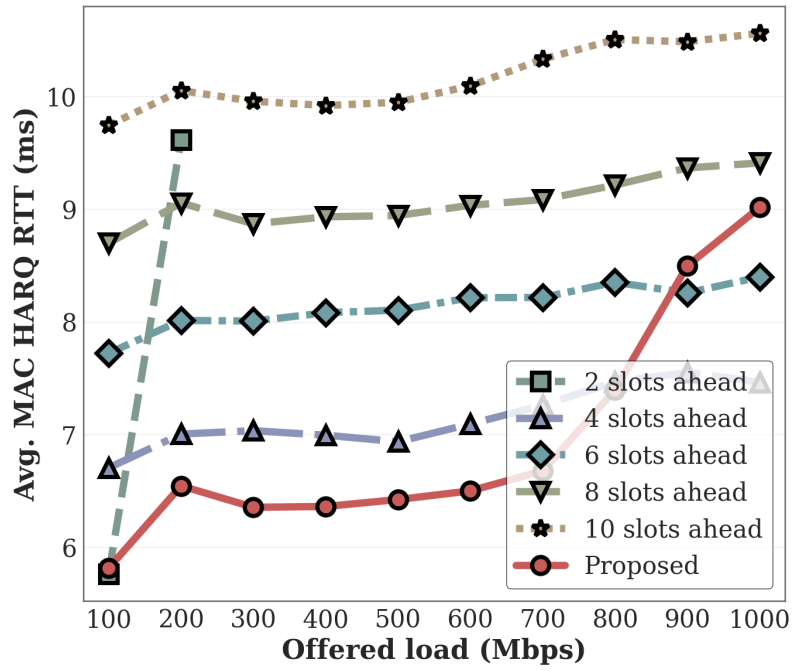
三、 情境 II：性能基準測試

在此情境中，我們評估了所提自適應定時機制相對於標準靜態提前時槽配置的基準性能。圖 4.3 展示了在 MAC 層延遲與系統吞吐量方面的性能評估。

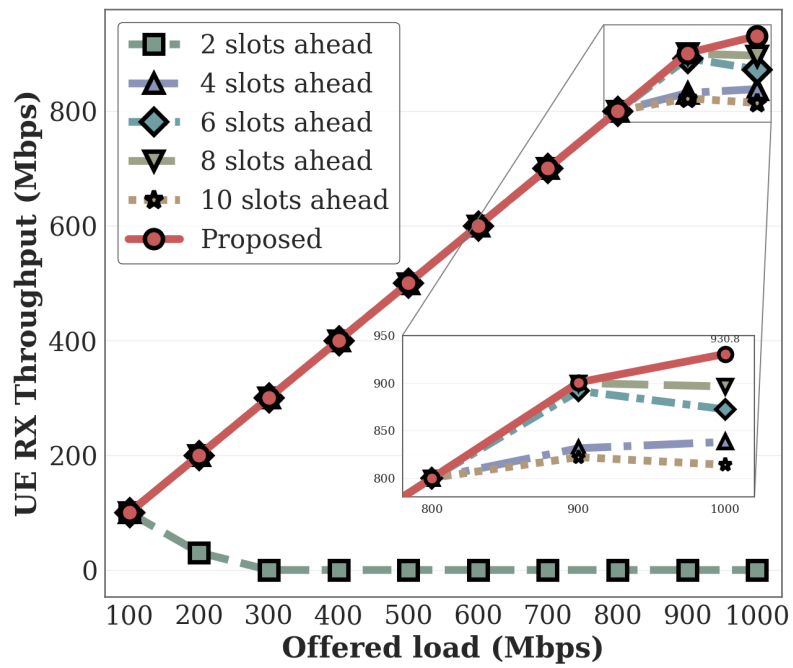
圖 4.3 展示了在不同負載 (Offered Load) 下的性能評估。所提演算法優先確保最佳吞吐量，在所有負載條件下皆能持續維持最佳的吞吐量表現。同時，在確保此最佳吞吐量的前提下，演算法也極大化地選擇最佳提前時槽，以優化 MAC 層 HARQ 往返時間 (RTT)，有效避免不必要的額外開銷。

四、 情境 III：網路擁塞下的穩定性

爲了測試系統的穩健性，我們利用 Linux TC 在 $T_2 - T_1$ 與 $T_5 - T_4$ 路段注入了人工網路擁塞，改變單向延遲 (OWD) 與抖動。如圖 4.4 所示，當延遲超過固定時窗時，靜態配置經歷了災難性的吞吐量下降與頻繁的連線重設；與此相對，所提自適應定時控制動態地擴展了提前時槽 (S_{ahead})。這種擴展允許 MAC 排程器預先觸發訊息，確保訊息儘管傳輸延遲增加，仍能在有效時窗內抵達 PNF。因此，即使在先前會導致完全同步失效的擁塞程度下，系統仍能維持穩定的吞吐量。



(a) MAC 延遲與可靠性



(b) 吞吐量優化

圖 4.3: 性能評估：MAC RTT 效率與系統吞吐量。

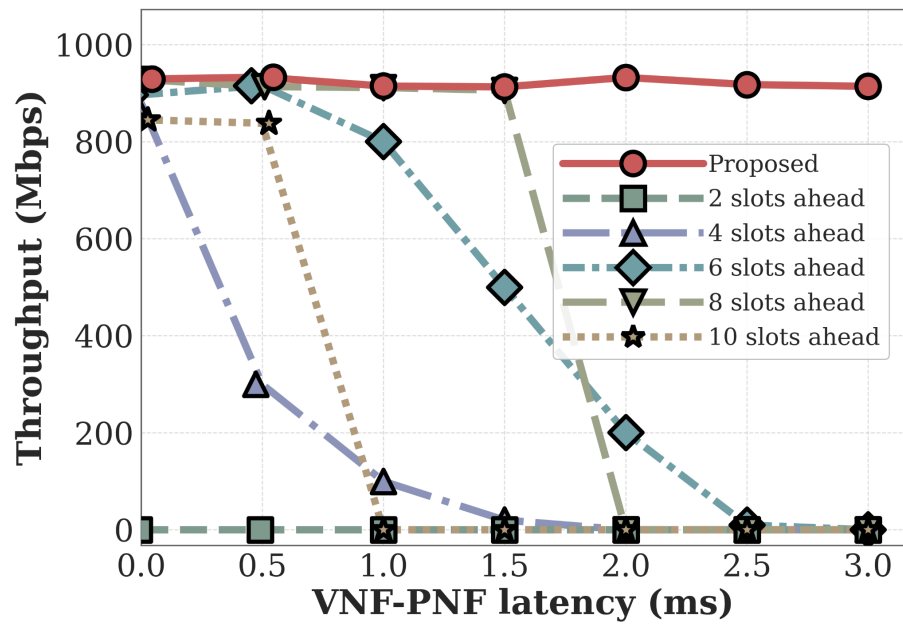


圖 4.4: 不同網路擁塞程度下的系統穩定性與吞吐量表現。

第五章 結論

本研究針對 O-RAN 開放式無線接取網路架構中的傳輸延遲問題，提出了一套自適應提前時槽 (Adaptive Slots Ahead) 機制，特別聚焦於採用 nFAPI 介面的 Option 6 功能分割架構。在非理想的封包交換網路環境中，傳統的靜態提前時槽設定無法有效應對不可預期的網路抖動與伺服器處理延遲，容易導致排程逾時與同步失敗。為了解決此一挑戰，本研究設計了一套基於指數加權移動平均 (EWMA) 模型的動態延遲管理演算法，透過閉迴路的反饋控制，持續追蹤並預測端到端的網路傳輸延遲特性。

透過整合 OpenAirInterface (OAI) 開源軟體與商用 Pegatron O-RU 設備的端到端 (End-to-End) 實體驗證平台，實驗結果證實了本研究所提之延遲管理架構能顯著提升系統的穩定度。在模擬網路壅塞的情境下，該機制能主動補償時間偏移，確保 MAC 層排程指令準確落入實體層的時序窗口內，徹底消除因時槽封包 (Slot Packet) 遺失所造成的連線中斷。相較於依賴昂貴硬體加速或嚴格硬體層級 PTP 同步的傳統方案，本研究所提出之基於純軟體的解決方案，在通用型商用現成 (COTS) 伺服器上展現了極高的部署彈性與成本效益，為未來 5G/6G 異質網路的商業化落地提供了具備高可靠度的關鍵技術基礎。

參 考 文 獻

- [1] O-RAN Alliance, “O-RAN Specifications.” [Online]. Available: <https://www.o-ran.org/specifications>, 2025.
- [2] 3GPP, “Study on new radio access technology: Radio access architecture and interfaces,” Tech. Rep. TR 38.801, 3rd Generation Partnership Project (3GPP), 2017. Version 14.0.0.
- [3] B. Agarwal, R. Irmer, D. Lister, and G.-M. Muntean, “Open ran for 6g networks: Architecture, use cases and open issues,” *IEEE Communications Surveys & Tutorials*, vol. 28, no. 3, pp. 2881–2924, 2025.
- [4] L. M. P. Larsen, A. Checko, and H. L. Christiansen, “A survey of the functional splits in 5g next-generation radio access networks,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2349–2372, 2019.
- [5] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, “A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2791–2833, 2023.
- [6] O-RAN Alliance, “O-ran fronthaul working group control, user and synchronization plane specification,” Tech. Rep. O-RAN.WG4.CUS.0-v11.00, O-RAN Alliance, 2023.
- [7] B. Khan, N. Nidhi, R. R. Paulo, A. Mihovska, and F. J. Velez, “Cost revenue trade-off for the 5g nr small cell network in the sub-6 ghz operating band,” in *2022 25th International Symposium on Wireless Personal Multimedia Communications (WPMC)*, pp. 64–69, 2022.
- [8] S. C. Forum, “5g fapi: Phy api specification,” Tech. Rep. SCF222, Small Cell Forum, 2021.
- [9] S. C. Forum, “5g network fapi (nfapi) specification,” Tech. Rep. SCF225, Small Cell Forum, 2020.
- [10] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, “nfapi: The standard interface for sdn-based 5g small cell networks,” *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.
- [11] X. Foukas *et al.*, “Computational resource consumption of 5g virtualized ran,” in *2021 IEEE International Conference on Communications (ICC)*, IEEE, 2021.
- [12] S.-Q. Lee and M.-S. Lee, “A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb,” in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*, IEEE, 2022.
- [13] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, “X5g: An open, programmable, multi-vendor, end-to-end, private 5g o-ran testbed with nvidia arc and openairinterface,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11305–11322, 2025.

- [14] A. S. Mohammed, K. S. Tharakan, H. A. Ammar, H. ElSawy, and H. S. Hassanein, "Fronthaul network planning for hierarchical and radio-stripes-enabled cf-mmimo in o-ran," *IEEE Transactions on Wireless Communications*, vol. 25, pp. 14781–14796, 2026.
- [15] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, A. Garcia-Saavedra, and X. Costa-Pérez, "Oranus: Latency-tailored orchestration via stochastic network calculus in 6g o-ran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 61–70, 2024.
- [16] Y. Chen, Y. T. Hou, W. Lou, J. H. Reed, and S. Kompella, "M3: A sub-millisecond scheduler for multi-cell mimo networks under c-ran architecture," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, 2022.
- [17] J. Lv, Y. Gao, Z. Ding, Y. Lin, X. You, G. Yang, and W. Dong, "Providing ue-level qos support by joint scheduling and orchestration for 5g vran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 51–60, 2024.
- [18] X. Wang *et al.*, "An open-source implementation of the 5g nr functional split option 6," in *2022 IEEE International Conference on Communications (ICC)*, pp. 1–6, 2022.
- [19] "Open5gs: Open source 5g core network." [Online]. Available: <https://open5gs.org/>, 2024.
- [20] OpenAirInterface, "OpenAirInterface Official Website." [Online]. Available: <https://openairinterface.org/>, 2025.
- [21] "Intel xeon gold 6226r processor (22m cache, 2.90 ghz)." [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/199347/intel-xeon-gold-6226r-processor-22m-cache-2-90-ghz/specifications.html>, 2025.
- [22] "Broadcom netxtreme bcm5719 quad-port 1gbe base-t adapter." [Online]. Available: <https://www.dell.com/zh-tw/shop/broadcom-5719-%E5%9B%9B%E9%80%A3%E6%8E%A5%E5%9F%A0-1gbe-base-t-%E9%85%8D%E6%8E%A5%E5%8D%A1-pcie-%E5%85%A8%E9%AB%98/apd/540-bdrj/%E7%B6%B2%E8%B7%AF%E8%A8%AD%E5%82%99>, 2026.
- [23] "Network interface controller." [Online]. Available: https://en.wikipedia.org/wiki/Network_interface_controller, 2024.
- [24] "Intel xeon gold 6433n processor (42m cache, 2.00 ghz)." [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/232148/intel-xeon-gold-6433n-processor-42m-cache-2-00-ghz/specifications.html>, 2025.
- [25] "Intel ethernet server adapter i210-t1." [Online]. Available: <https://www.intel.com.tw/content/www/tw/zh/products/sku/68668/intel-ethernet-server-adapter-i210t1/specifications.html>, 2026.

- [26] 3GPP, “User equipment (ue) radio transmission and reception,” Technical Specification (TS) 38.101, 3rd Generation Partnership Project (3GPP), Sept. 2024. Version 16.17.0.
- [27] O-RAN Working Group 1, *O-RAN Architecture Description*. O-RAN Alliance, June 2025. Rev. 14.00.
- [28] O-RAN Working Group 2, *Non-RT RIC Architecture*. O-RAN Alliance, February 2022. O-RAN.WG2.Non-RT-RIC-ARCH-v07.00.
- [29] “Android debug bridge (adb) command-line tool.” [Online]. Available: <https://developer.android.com/tools/adb>, 2024.
- [30] E. E. S. Network), “iperf3 - the ultimate speed test tool for tcp, udp and sctp.” [Online]. Available: <https://iperf.fr/>, 2025.
- [31] The Tcpdump Group, “tcpdump.” [Online]. Available: <https://www.tcpdump.org/>, 2025.
- [32] R. Cochran, “Linux ptp v3.1.1: Precision time protocol (ptp) implementation for linux.” [Online]. Available: <https://github.com/richardcochran/linuxptp/releases/tag/v3.1.1>, 2023.
- [33] TSN Community, “Time synchronization - time sensitive networking (tsn).” [Online]. Available: <https://tsn.readthedocs.io/timesync.html#checking-clocks-synchronization>, 2024.
- [34] DPDK Project, “Data plane development kit (dpdk) v20.11.9.” [Online]. Available: <https://doc.dpdk.org/guides-20.11/>, 2023. Long Term Support (LTS) Release.